

set_post_filter

REFERENCE

API consumer

add, delete, or modify a post-filter rule on the reader

Overview

`set_post_filter` **configures a post-filter** — a rule that includes or excludes scanned RFID tag data before it is reported to a data endpoint. Each call operates on **one** filter, chosen by `postFilterPayload.operation` (`ADD` / `DELETE` / `MODIFY`): the rule matches tag data by pattern (prefix, suffix, or regex) and either reports (`INCLUDE`) or drops (`EXCLUDE`) it, scoped to a data endpoint. It is a state-changing **control-plane (CTRL)** command; read the current filters back with `get_post_filter`.

This is a **CTRL-endpoint command**, published on the **control** endpoint

(`<tenantId>/CTRL/clients/cmd/<serial>`) and answered only there (not on the

[MGMT/MDM](#) channel). Returning a response requires an **active CTRL endpoint**

(`config_endpoint`, `epType: CTRL`, `activate: true`). Also: `dataEpType` is the **enum**

`DATA_EP1` / `DATA_EP2`, **not** the endpoint *name* — sending an endpoint name returns [code 23](#) (see [Request Semantics](#)).

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	STATE-CHANGING (adds/removes a filter)
Reversibility	REVERSIBLE (an <code>ADD</code> is undone by a <code>DELETE</code> on the same <code>dataEpType</code>)
Supported Operations	<code>ADD</code> (new), <code>DELETE</code> (existing), <code>MODIFY</code> (existing)
Required Request Fields	<code>command</code> , <code>requestId</code> (<code>postFilterPayload</code> is optional in the contract; send it with the rule)
Applies To	Endpoint: Control (CTRL). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)

Property	Value
Related	<code>get_post_filter</code> , <code>set_operating_mode</code> , <code>config_endpoint</code>

When to Use

Use `set_post_filter` to control which scanned tag data reaches a data endpoint:

- **Include only matching tags** — `reportOperation: INCLUDE` reports only tags whose data matches; everything else is dropped.
- **Exclude noisy tags** — `reportOperation: EXCLUDE` drops matching tags from reporting.
- **Match by prefix, suffix, or regex** — choose `matchPatternMethod` to fit the tag-data pattern.
- **Manage the rule set** — `ADD` to install, `DELETE` to remove, `MODIFY` to update; verify with `get_post_filter`.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/CTRL/clients/cmnd/<serial></code>	On the control endpoint. Requires an active CTRL endpoint .
Response	Reader → App	<code><tenantId>/CTRL/clients/resp/<serial></code>	Correlated reply; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Control commands are answered **only on the CTRL endpoint**.

Post-filters act on **tag data** (post-singulation), distinct from the Gen2 `select` pre-filters in `set_operating_mode` that act on the air interface. If control commands time out, confirm a CTRL endpoint is active and activate one with `config_endpoint` if needed.

See [Topics and Channels](#).

Key Fields

- `postFilterPayload.operation` (request) — `ADD` / `DELETE` / `MODIFY`; determines the action on the rule set.
- `dataEpType` (request) — the data-endpoint enum (`DATA_EP1` / `DATA_EP2`) the filter binds to — ⚠ not the endpoint name (see [Request Semantics](#)).
- `matchPattern` + `matchPatternMethod` (request) — what to match and how (`PREFIX` / `SUFFIX` / `REGEX`).
- `reportOperation` (request) — `INCLUDE` (report matches) or `EXCLUDE` (drop matches).
- `response.code` (response) — read this to determine success vs failure. The contract requires it; note the reported success-path omission (see [Success without a code](#)). See [Response Codes](#).

Request Payload

The envelope is `command`, `requestId`, and the single-rule `postFilterPayload`. Pick a **match method** below; the Example and Schema update together. Use the **Schema** tab for the full field reference and the **Example** tab for the concrete payload.

Post filter scenario

Filter tags using prefix



Request payload

Example: ADD, prefix

```
1  {
2    "command" : "set_post_filter" ,
3    "requestId" : "abc123" ,
4    "postFilterPayload" : {
5      "operation" : "ADD" ,
6      "dataEpType" : "DATA_EP1" ,
7      "matchPattern" : "FFFFBBBBBA500" ,
8      "matchPatternMethod" : "PREFIX" ,
9      "reportOperation" : "INCLUDE"
10   }
11 }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	co mm an d *	string	—	set_post_filter	Fixed command name.
2	re qu es tI d *	string	—	abc123	A unique identifier for the request; echoed back in the response.
3	post Filt erPa yloa d	object	—	—	The single post-filter rule to add, delete, or modify. Optional in the contract, as is every field inside it, but a request without it carries no rule to apply.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
4	operation	enum	ADD DELETE MODIFY	ADD	ADD a new filter, DELETE an existing one, or MODIFY one. See Request Semantics .
5	dataEndpointType	enum	DATA_EP1 DATA_EP2	DATA_EP1	The data-endpoint TYPE the filter binds to. ⚠ This is the enum value DATA_EP1/DATA_EP2, not the endpoint name (see Request Semantics) — a wrong value returns code 23 . DATA_EP2 targets the DATA2 endpoint, which is currently not supported.
6	matchPattern	string	—	FFFFBBBBBA500	The value matched against scanned tag data. For PREFIX/SUFFIX: hex digits only, even count. For REGEX: a regular expression. See Request Semantics.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
7	m a t c h P a t t e r n M e t h o d	enum	PREFIX SUFFIX REGEX	REGEX	How the pattern is matched — beginning (PREFIX), end (SUFFIX), or regular expression (REGEX).
8	r e p o r t O p e r a t i o n	enum	INCLUDE EXCLUDE	INCLUDE	INCLUDE reports only matching tag data; EXCLUDE drops matching tag data from reporting.

Request Semantics

Modes. `postFilterPayload.operation` selects the action; each call handles one rule:

- `ADD` — install a new post-filter for the given `dataEpType`.
- `DELETE` — remove the post filter on a data endpoint. Send `operation: DELETE` with `dataEpType` alone; `matchPattern`, `matchPatternMethod` and `reportOperation` are not needed. Undoes a prior `ADD` on the same `dataEpType`.
- `MODIFY` — update an existing filter.

Confirm the resulting rule set with `get_post_filter`, which returns the stored `postFilter[]` (or nothing when none remain).

- `dataEpType` **is an enum, not the endpoint name.** `dataEpType` must be the enum value `DATA_EP1` or `DATA_EP2` — **not** the endpoint *name* you gave a data endpoint in `config_endpoint` (e.g. `DATA1_EP`). Sending the endpoint name is an enum violation and the device rejects it with [code 23](#) ("Invalid enum value"). Map your `DATA1` endpoint to `DATA_EP1` when setting a filter; `DATA_EP2` targets the `DATA2` endpoint, which is currently not supported.
- `matchPattern` **rules follow the match method.** For `PREFIX` and `SUFFIX` methods, `matchPattern` must be **hexadecimal digits only, with an even count** — the contract states this rule in the field description but does not enforce it with a pattern; for `REGEX` it is a free-form regular expression.
- `PREFIX` **post-filters and Gen2 `select` pre-filters are mutually exclusive.** A `PREFIX` post-filter precludes using Gen2 `select` pre-filters at the same time.

Response Payload

`set_post_filter` returns a **status-only** envelope — `command`, `requestId`, `apiVersion`, and `response`. Confirm with `get_post_filter` — there is no filter echo.

Response payload

```

1  {
2      "command" : "set_post_filter" ,
3      "requestId" : "abc123" ,
4      "apiVersion" : "V1.1" ,
5      "response" : {
6          "code" : 0 ,
7          "description" : "Success"
8      }
9  }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	set_post_filter	Echoes the executed command.
2	requestId *	string	—	abc123	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	response *	object	—	—	Status-only result — set_post_filter echoes no configuration.
5	code *	integer	0–101	0	0 = Success; non-zero = error. Required by the contract. Readers have been reported to omit it on success — see Response Payload and Response Codes below.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
6	d e s c r i p t i o n	string	—	Success	Human-readable status.

The contract requires `response.code`. Readers have been reported to omit it on a successful reply, returning just `response: { "description": "Success" }` — treat that as success too. See [Success without a code](#).

As reported, a successful set or delete can arrive with `code` omitted. This form does not validate against the contract; the reply with `code: 0` above is the documented form:

Response — success reported without code (not accepted by the contract)

```

1  {
2    "command" : "set_post_filter" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "description" : "Success"
7    }
8  }
```

Response Codes

`set_post_filter` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the filter was added, deleted, or modified. (Readers have been reported to omit <code>code</code> on this reply — see above.)
23	Invalid enum value (<code>IOT_ERROR_INVALID_ENUM</code>) — a field (<code>operation</code> , <code>dataEpType</code> , <code>matchPatternMethod</code> , Or <code>reportOperation</code>) is outside its allowed set. Commonly triggered by sending an endpoint <i>name</i> (e.g. <code>DATA1_EP</code>) as <code>dataEpType</code> instead of the enum <code>DATA_EP1</code> — see Request Semantics.
101	Default error — generic catch-all.

Examples

Publish `set_post_filter` over the CTRL endpoint; the device replies with the status envelope under [Response Payload](#). The **Post filter scenario** dropdown under [Request Payload](#) shows each contract example: an `ADD` by prefix, regex and suffix, and a `DELETE`. In the regex example, `matchPattern` is free-form; the hex-digit rule applies only to prefix and suffix patterns.

Sending an endpoint **name** (not the enum) as `dataEpType` is rejected with [code](#) `23`:

Response — invalid enum value (code 23)

```

1  {
2      "command" : "set_post_filter" ,
3      "requestId" : "abc123" ,
4      "apiVersion" : "V1.1" ,
5      "response" : {
6          "code" : 23 ,
7          "description" : "Invalid enum value"
8      }
9  }
```

Notes and Caveats

- Post-filters vs pre-filters.** `set_post_filter` configures **post-filters** — rules applied to scanned **tag data** before it reaches a data endpoint (`INCLUDE` / `EXCLUDE`). These are separate from the Gen2 `select` **pre-filters** in `set_operating_mode` , which filter on the air interface during inventory.

`set_post_filter` is **reversible** — an `ADD` is undone by a `DELETE` on the same `dataEpType`. It carries no credentials (match patterns are tag-data hex/regex strings). After setting a filter, **verify with** `get_post_filter` — the setter's response has no configuration echo. For the concept-level view, see [Tag Filtering](#).

Related:  `get_post_filter` ·  `set_operating_mode` ·  `config_endpoint` ·  [Tag Filtering](#) ·  [Response Codes](#)